The Ghost Architect

TGA — Generative System Engineering Tool

Talk about your system. Watch it come to life.

TGA listens to architecture discussions — voice or text — and automatically generates live architecture diagrams, extracted service maps, manifesto documentation, and a ready-to-run project scaffold.

1. What Does TGA Do?

In a typical engineering meeting, engineers describe a system verbally. After the meeting, someone must manually draw diagrams, write documentation, and scaffold the project. TGA automates that first draft.

TGA produces five outputs automatically:

- Live architecture diagram (Mermaid.js)
- Extracted entities — services, databases, APIs, queues, relationships
- Architect's Manifesto — professional Markdown documentation
- Starter codebase scaffold with Dockerfiles and Docker Compose
- Browser microphone recording for voice-to-diagram generation

2. High-Level Processing Flow

Every piece of text or audio passes through this pipeline:

Step 1	User enters text or uploads / records audio in the Streamlit browser UI.
Step 2	FastAPI backend (main.py) receives the input and coordinates all processing.
Step 3	Audio is transcribed to text chunks using Faster-Whisper (speech-to-text).
Step 4	The local LLM reads each chunk and extracts architecture entities as JSON.
Step 5	Shared architecture state is updated: services, databases, APIs, queues, relationships.
Step 6	Mermaid diagram code is generated and rendered live in the browser.
Step 7	Blueprint JSON, Architect's Manifesto, and project scaffold are generated on demand.

3. Technology Stack

Area	Technology	Purpose
Frontend	Streamlit	Browser UI — text/audio input, diagram, entities, export
Backend	FastAPI	API server — receives input, processes data, exposes state
Live updates	Autorefresh + /state	Polls every second for live diagram updates
Transcription	Faster-Whisper	Converts audio to text chunks on CPU
Browser record	MediaRecorder API	Records microphone inside the browser
LLM inference	llama.cpp / llama-cpp-python	Runs quantized GGUF model locally on CPU
Model file	models/llm/model.gguf	Local quantized language model
Diagram	Mermaid.js	Renders architecture graph in browser
Scaffold	Python file generation	Creates service folders and boilerplate
Packaging	ZIP	Downloads project as generated_project.zip

4. AI / ML Classification

TGA uses four layers of AI technology:

Machine Learning (ML)	Faster-Whisper detects speech and converts it to text. The local LLM reads architecture descriptions and extracts structured entities.
Deep Learning (DL)	Whisper / Faster-Whisper is a deep learning speech recognition model. The GGUF LLM is a deep learning transformer model.
Natural Language Processing (NLP)	Meeting text is analysed to find services, databases, APIs, and relationships. The model ignores small talk and extracts architecture intent.
Generative AI	The LLM generates structured JSON from natural meeting language, writes the Architect's Manifesto, and drives the project scaffold generator.

5. Key Files & Folders

Path	Purpose
app.py	Streamlit frontend — browser UI
main.py	FastAPI backend — orchestrates all processing
scripts/audio_pipeline.py	Speech-to-text using Faster-Whisper
scripts/architecture_extraction_pipeline.py	Core LLM extraction — entities & relationships
scripts/mermaid_generator.py	Converts architecture state to Mermaid diagram code
scripts/blueprint_generator.py	Creates output/blueprint.json (folder plan)
scripts/scaffold_generator.py	Physically creates the generated project folder
scripts/manifesto_generator.py	LLM generates the Architect's Manifesto Markdown
models/llm/model.gguf	Local quantized LLM file (CPU-friendly)
output/	Generated JSON and Markdown outputs
generated_project/	Generated codebase scaffold
generated_project.zip	Downloadable scaffold ZIP

6. Backend API Endpoints

Endpoint	Method	Purpose
/set-context	POST	Save optional project context
/start-session-text	POST	Process pasted architecture text
/upload-file	POST	Upload MP3/WAV/M4A/WEBM audio
/start-session	POST	Transcribe audio and extract architecture
/stop-session	POST	Stop current processing
/reset-session	POST	Clear extracted state
/state	GET	Return latest architecture state (polled every 1 s)
/status	GET	Return processing stage, chunk count, latest chunk
/generate-scaffold	POST	Generate blueprint, manifesto, scaffold, ZIP
/open-folder	GET	Open generated project folder locally

7. Runtime Flows

7a — Text to Diagram

Step 1	User opens Streamlit at http://localhost:8501 and pastes architecture text.
Step 2	Click Generate Diagram → app.py calls POST /start-session-text.
Step 3	main.py splits text into chunks and sends each to the extraction pipeline.
Step 4	Local LLM extracts services, databases, APIs, queues, and relationships as JSON.
Step 5	State saved to output/architecture_state.json and kept in memory.
Step 6	app.py polls GET /state every second and rebuilds the Mermaid diagram.
Step 7	Streamlit renders the live updated architecture diagram.

7b — Uploaded Audio to Diagram

Step 1	User uploads MP3 / WAV / M4A / WEBM in Streamlit sidebar.
Step 2	app.py sends file to POST /upload-file → saved to input/uploads/.
Step 3	User clicks Process Audio → app.py calls POST /start-session.
Step 4	Faster-Whisper transcribes the audio into text chunks.
Step 5	Chunks enter the LLM extraction pipeline — same as text flow.
Step 6	Diagram and entity list update live as Streamlit polls /state.

7c — Browser Voice Recording to Diagram

Step 1	User clicks Start Recording → browser requests microphone permission.
Step 2	MediaRecorder API records speech as .webm audio in the browser.
Step 3	User clicks Stop & Process → browser sends .webm to POST /upload-file.
Step 4	main.py saves browser_recording.webm to input/uploads/.
Step 5	Faster-Whisper transcribes → LLM extraction pipeline processes chunks.
Step 6	Diagram updates live every second via /state polling.

7d — Generate Project

Step 1	User clicks Generate Project → app.py calls POST /generate-scaffold.

Step 2	blueprint_generator.py creates output/blueprint.json (folder plan).
Step 3	manifesto_generator.py asks LLM to write ARCHITECTS_MANIFESTO.md.
Step 4	scaffold_generator.py creates generated_project/ with all service folders.
Step 5	main.py zips everything into generated_project.zip.
Step 6	Streamlit shows the download button for the ZIP file.

8. Example Text Input

Paste this into the Streamlit text box to see TGA in action:

```
Frontend talks to API gateway using HTTPS
API gateway routes login requests to auth service
Auth service stores user accounts in PostgreSQL
API gateway routes payment requests to payment service
Payment service calls Stripe API using REST
Payment service stores transactions in PostgreSQL
Auth service publishes user-created events to Redis queue
Notification service consumes events from Redis queue
Notification service sends emails using SendGrid API
```

Expected extracted entities:

- Services: frontend, api gateway, auth service, payment service, notification service
- Databases: PostgreSQL
- External APIs: Stripe API, SendGrid API
- Message Queue: Redis queue
- Relationships: all service-to-service and service-to-data-store connections

9. How To Run Locally

Open two terminal windows from the project root:

```
cd D:\Task\TGA

# Terminal 1 — Backend
uvicorn main:app --reload

# Terminal 2 — Frontend
streamlit run app.py
```

Streamlit UI	http://localhost:8501
FastAPI Backend	http://localhost:8000

10. Strengths & Current Limitations

Strengths

- Runs 100% locally — no cloud API required
- CPU-friendly quantized GGUF model
- Supports text input and uploaded audio files
- Live architecture diagram that updates every second
- Generates project scaffold with Dockerfiles and Docker Compose
- Generates Architect's Manifesto documentation
- Downloadable ZIP of the generated project

Limitations

- Live microphone streaming not fully connected in the main UI
- Extraction accuracy depends heavily on the GGUF model quality
- LLM JSON output can sometimes be invalid (no retry yet)
- Blueprint generation is basic (service-folder level only)
- Generated code is boilerplate — not production-ready business logic
- No session persistence — state is in memory and JSON files
- No authentication/security on backend endpoints

11. Improvement Roadmap

Better Model Reliability	Stronger instruction-tuned GGUF model · JSON schema validation · Auto-retry on invalid output · Confidence scores for extracted entities
Better Audio Pipeline	Live microphone streaming · Long audio chunking with timestamps · Raw transcript history panel
Better Extraction	Detect load balancers, caches, object storage · Smarter name merging · Synonym understanding (DB / database / Postgres) · Few-shot prompt examples
Better Diagrams	Node types for caches, queues, external APIs · Colour coding by type · Subgraphs for frontend / backend / data / external · D3.js option
Better Scaffold	Service-specific route names · Auto Docker Compose dependencies · Redis / DB containers auto-added · Jinja2 templates · Pytest stubs
Better UX	Show transcript chunks live · Expose model errors in UI · Session history · Export Mermaid SVG, manifesto, blueprint, transcript

12. Mental Model (For Anyone)

Think of TGA as a smart factory with six machines:

Step 1	Meeting words (voice or text) go in.
Step 2	Speech-to-text machine turns audio into readable text.
Step 3	Architect AI reads the text and writes down system parts as structured data.
Step 4	Diagram machine draws the architecture graph automatically.
Step 5	Documentation machine writes the Architect's Manifesto.
Step 6	Scaffold machine creates starter code folders ready to build on.